ID NO:2400030040

Name: N. Sri Aravind.

#SKILL - 3 -> Working with HQL - Sorting, Pagination & Aggregates

Prerequisites:

- **Basic knowledge of Hibernate**
- **Basic idea of HQL syntax**

In this lab, you will extend the Hibernate module created in Skill 2 and perform advanced data retrieval using HQL, including sorting, pagination, filtering, and aggregate functions. These operations help in retrieving structured and meaningful data from the database using Hibernate.

Tasks:

1. Use the Product entity created in Skill 2.
2. Add 5–8 additional Product records into the database.
3. Write HQL queries to retrieve all products sorted by price: a. Ascending order b. Descending order
4. Write an HQL query to sort products by quantity (highest first).
5. Implement pagination using HQL to display: a. First 3 products b. Next 3 products
6. Write HQL queries for aggregate operations: a. Count total number of products b. Count products where quantity > 0 c. Count products grouped by description d. Find minimum and maximum price
7. Write an HQL query using GROUP BY to group products by description.
8. Write an HQL query using WHERE to filter products within a price range.
9. Write HQL queries using LIKE such as: a. Names starting with certain letters b. Names ending with certain letters c. Names containing a pattern anywhere (substring) d. Names with an exact character length
10. Push the Hibernate project to GitHub

1)Product.java

```
package com.example;

import javax.persistence.*;

@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private int id;

    private String name;
    private String description;
    private double price;
    private int quantity;

    public Product() {}

    public Product(String name, String description, double price, int quantity) {
        this.name = name;
        this.description = description;
        this.price = price;
        this.quantity = quantity;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public String getDescription() { return description; }
    public double getPrice() { return price; }
```

```
    public int getQuantity() { return quantity; }


    public void setPrice(double price) { this.price = price; }
    public void setQuantity(int quantity) { this.quantity = quantity; }
}
```

2)MainApp.java

```
package com.example;


import java.util.List;
import java.util.Scanner;


import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;
import org.hibernate.query.Query;


public class MainApp {


    static SessionFactory factory =
            new Configuration().configure().buildSessionFactory();
    static Scanner sc = new Scanner(System.in);


    public static void main(String[] args) {


        int choice;


        do {
            System.out.println("\n===== PRODUCT MANAGEMENT (HQL LAB) =====");
            System.out.println("1. Insert Product");
            System.out.println("2. Insert Sample Products");
```

```
System.out.println("3. View Product By ID");
System.out.println("4. Sort Products By Price (ASC)");
System.out.println("5. Sort Products By Price (DESC)");
System.out.println("6. Sort Products By Quantity (High → Low)");
System.out.println("7. Pagination (First 3 / Next 3)");
System.out.println("8. Aggregate Functions");
System.out.println("9. Filter By Price Range");
System.out.println("10. LIKE Queries");
System.out.println("11. Exit");
System.out.print("Enter choice: ");

choice = sc.nextInt();

switch (choice) {
case 1:
    insertProduct();
    break;

case 2:
    insertSampleProducts();
    break;

case 3:
    viewProduct();
    break;

case 4:
    sortByPriceAsc();
    break;

case 5:
    sortByPriceDesc();
    break;
```

```
        case 6:
            sortByQuantity();
            break;

        case 7:
            paginationDemo();
            break;

        case 8:
            aggregateOperations();
            break;

        case 9:
            filterByPriceRange();
            break;

        case 10:
            likeQueries();
            break;

        case 11:
            factory.close();
            System.out.println("Exiting Application...");
            break;

        default:
            System.out.println("Invalid Choice!");
    }



    } while (choice != 11);
}
```

```
// 1. INSERT PRODUCT
static void insertProduct() {
    Session session = factory.openSession();
    Transaction tx = session.beginTransaction();

    sc.nextLine();
    System.out.print("Enter Name: ");
    String name = sc.nextLine();

    System.out.print("Enter Description: ");
    String desc = sc.nextLine();

    System.out.print("Enter Price: ");
    double price = sc.nextDouble();

    System.out.print("Enter Quantity: ");
    int qty = sc.nextInt();

    Product p = new Product(name, desc, price, qty);
    session.save(p);

    tx.commit();
    session.close();
    System.out.println("Product Inserted Successfully!");
}

// 2. INSERT SAMPLE PRODUCTS (5–8 RECORDS)
static void insertSampleProducts() {
    Session session = factory.openSession();
    Transaction tx = session.beginTransaction();

    session.save(new Product("Laptop", "Electronics", 55000, 10));
```

```
    session.save(new Product("Mouse", "Electronics", 500, 50));
    session.save(new Product("Keyboard", "Electronics", 1200, 30));
    session.save(new Product("Chair", "Furniture", 3500, 15));
    session.save(new Product("Table", "Furniture", 8000, 5));
    session.save(new Product("Bottle", "Accessories", 300, 0));

    tx.commit();
    session.close();
    System.out.println("Sample Products Inserted!");
}

// 3. VIEW PRODUCT BY ID
static void viewProduct() {
    Session session = factory.openSession();

    System.out.print("Enter Product ID: ");
    int id = sc.nextInt();

    Product p = session.get(Product.class, id);

    if (p != null) {
        System.out.println(p.getId() + " | " +
                p.getName() + " | " +
                p.getDescription() + " | " +
                p.getPrice() + " | " +
                p.getQuantity());
    } else {
        System.out.println("Product Not Found!");
    }

    session.close();
}
```

```
// 4. SORT BY PRICE ASC
static void sortByPriceAsc() {
    Session session = factory.openSession();

    Query<Product> q =
            session.createQuery("FROM Product p ORDER BY p.price ASC",
                    Product.class);

    q.list().forEach(MainApp::printProduct);

    session.close();
}

// 5. SORT BY PRICE DESC
static void sortByPriceDesc() {
    Session session = factory.openSession();

    Query<Product> q =
            session.createQuery("FROM Product p ORDER BY p.price DESC",
                    Product.class);

    q.list().forEach(MainApp::printProduct);

    session.close();
}

// 6. SORT BY QUANTITY (HIGH → LOW)
static void sortByQuantity() {
    Session session = factory.openSession();

    Query<Product> q =
            session.createQuery("FROM Product p ORDER BY p.quantity DESC",
                    Product.class);
```

```
    q.list().forEach(MainApp::printProduct);

    session.close();
}

// 7. PAGINATION
static void paginationDemo() {
    Session session = factory.openSession();

    System.out.println("\n--- First 3 Products ---");
    Query<Product> q1 =
            session.createQuery("FROM Product", Product.class);
    q1.setFirstResult(0);
    q1.setMaxResults(3);
    q1.list().forEach(MainApp::printProduct);

    System.out.println("\n--- Next 3 Products ---");
    Query<Product> q2 =
            session.createQuery("FROM Product", Product.class);
    q2.setFirstResult(3);
    q2.setMaxResults(3);
    q2.list().forEach(MainApp::printProduct);

    session.close();
}

// 8. AGGREGATE FUNCTIONS
static void aggregateOperations() {
    Session session = factory.openSession();

    Long total =
            session.createQuery("SELECT COUNT(p) FROM Product p",
```

```
                Long.class).getSingleResult();
    System.out.println("Total Products: " + total);

    Long available =
        session.createQuery(
                "SELECT COUNT(p) FROM Product p WHERE p.quantity > 0",
                Long.class).getSingleResult();
    System.out.println("Products with Quantity > 0: " + available);

    Object[] minMax =
        (Object[]) session.createQuery(
                "SELECT MIN(p.price), MAX(p.price) FROM Product p")
                .getSingleResult();

    System.out.println("Min Price: " + minMax[0]);
    System.out.println("Max Price: " + minMax[1]);

    System.out.println("\nProducts Grouped By Description:");
    List<Object[]> list =
        session.createQuery(
                "SELECT p.description, COUNT(p) FROM Product p GROUP BY p.description")
                .getResultList();

    for (Object[] row : list) {
        System.out.println(row[0] + " → " + row[1]);
    }

    session.close();
}

// 9. FILTER BY PRICE RANGE
static void filterByPriceRange() {
    Session session = factory.openSession();
```

```
    Query<Product> q =
        session.createQuery(
            "FROM Product p WHERE p.price BETWEEN :min AND :max",
            Product.class);

    q.setParameter("min", 1000.0);
    q.setParameter("max", 10000.0);

    q.list().forEach(MainApp::printProduct);

    session.close();
}

// 10. LIKE QUERIES
static void likeQueries() {
    Session session = factory.openSession();

    System.out.println("\nNames starting with 'L':");
    session.createQuery(
        "FROM Product p WHERE p.name LIKE 'L%'", Product.class)
        .list().forEach(MainApp::printProduct);

    System.out.println("\nNames ending with 'r':");
    session.createQuery(
        "FROM Product p WHERE p.name LIKE '%r'", Product.class)
        .list().forEach(MainApp::printProduct);

    System.out.println("\nNames containing 'top':");
    session.createQuery(
        "FROM Product p WHERE p.name LIKE '%top%'", Product.class)
        .list().forEach(MainApp::printProduct);
```

```
        System.out.println("\nNames with length = 5:");
        session.createQuery(
                "FROM Product p WHERE LENGTH(p.name) = 5", Product.class)
                .list().forEach(MainApp::printProduct);

        session.close();
    }

    // COMMON PRINT METHOD
    static void printProduct(Product p) {
        System.out.println(p.getId() + " | " +
                p.getName() + " | " +
                p.getDescription() + " | " +
                p.getPrice() + " | " +
                p.getQuantity());
    }
```

3)hibernate.cfg.xml

```
<hibernate-configuration>
<session-factory>
 <property name="hibernate.dialect">org.hibernate.dialect.MySQL8Dialect</property>
 <property name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
 <property name="hibernate.connection.url">jdbc:mysql://localhost:3306/skill3</property>
 <property name="hibernate.connection.username">root</property>
 <property name="hibernate.connection.password">Aravind@17</property>
 <property name="hibernate.hbm2ddl.auto">update</property>
 <property name="show_sql">true</property>
 <mapping class="com.example.Product"/>
</session-factory>
</hibernate-configuration>
```

Output:

```
===== PRODUCT MANAGEMENT (HQL LAB) =====
1. Insert Product
2. Insert Sample Products
3. View Product By ID
4. Sort Products By Price (ASC)
5. Sort Products By Price (DESC)
6. Sort Products By Quantity (High → Low)
7. Pagination (First 3 / Next 3)
8. Aggregate Functions
9. Filter By Price Range
10. LIKE Queries
11. Exit
Enter choice:
```

```
===== PRODUCT MANAGEMENT (HQL LAB) =====
1. Insert Product
2. Insert Sample Products
3. View Product By ID
4. Sort Products By Price (ASC)
5. Sort Products By Price (DESC)
6. Sort Products By Quantity (High → Low)
7. Pagination (First 3 / Next 3)
8. Aggregate Functions
9. Filter By Price Range
10. LIKE Queries
11. Exit
Enter choice: 1
Enter Name:  Laptop
Enter Description:  HP
Enter Price:  50000
Enter Quantity:  1
Hibernate: select next_val as id_val from hibernate_sequence for update
Hibernate: update hibernate_sequence set next_val= ? where next_val=?
Hibernate: insert into products (description, name, price, quantity, id) values (?, ?, ?, ?, ?)
Product Inserted Successfully!
```

```
===== PRODUCT MANAGEMENT (HQL LAB) =====
1. Insert Product
2. Insert Sample Products
3. View Product By ID
4. Sort Products By Price (ASC)
5. Sort Products By Price (DESC)
6. Sort Products By Quantity (High → Low)
7. Pagination (First 3 / Next 3)
8. Aggregate Functions
9. Filter By Price Range
10. LIKE Queries
11. Exit
Enter choice: 2
Hibernate: select next_val as id_val from hibernate_sequence for update
Hibernate: update hibernate_sequence set next_val= ? where next_val=?
Hibernate: select next_val as id_val from hibernate_sequence for update
Hibernate: update hibernate_sequence set next_val= ? where next_val=?
Hibernate: select next_val as id_val from hibernate_sequence for update
Hibernate: update hibernate_sequence set next_val= ? where next_val=?
Hibernate: select next_val as id_val from hibernate_sequence for update
Hibernate: update hibernate_sequence set next_val= ? where next_val=?
Hibernate: select next_val as id_val from hibernate_sequence for update
Hibernate: update hibernate_sequence set next_val= ? where next_val=?
Hibernate: select next_val as id_val from hibernate_sequence for update
Hibernate: update hibernate_sequence set next_val= ? where next_val=?
Hibernate: insert into products (description, name, price, quantity, id) values (?, ?, ?, ?, ?)
Hibernate: insert into products (description, name, price, quantity, id) values (?, ?, ?, ?, ?)
Hibernate: insert into products (description, name, price, quantity, id) values (?, ?, ?, ?, ?)
Hibernate: insert into products (description, name, price, quantity, id) values (?, ?, ?, ?, ?)
Hibernate: insert into products (description, name, price, quantity, id) values (?, ?, ?, ?, ?)
Hibernate: insert into products (description, name, price, quantity, id) values (?, ?, ?, ?, ?)
Sample Products Inserted!
```

```
===== PRODUCT MANAGEMENT (HQL LAB) =====
1. Insert Product
2. Insert Sample Products
3. View Product By ID
4. Sort Products By Price (ASC)
5. Sort Products By Price (DESC)
6. Sort Products By Quantity (High → Low)
7. Pagination (First 3 / Next 3)
8. Aggregate Functions
9. Filter By Price Range
10. LIKE Queries
11. Exit
Enter choice: 3
Enter Product ID:  101
Hibernate: select product0_.id as id1_0_0_, product0_.description as descript2_0_0_, product0_.name as name3_0_0_, product0_.price as price4_0_0_, product
Product Not Found!
```

```
===== PRODUCT MANAGEMENT (HQL LAB) =====
1. Insert Product
2. Insert Sample Products
3. View Product By ID
4. Sort Products By Price (ASC)
5. Sort Products By Price (DESC)
6. Sort Products By Quantity (High → Low)
7. Pagination (First 3 / Next 3)
8. Aggregate Functions
9. Filter By Price Range
10. LIKE Queries
11. Exit
Enter choice: 4
Hibernate: select product0_.id as id1_0_, product0_.description as descript2_0_, product0_.name as name3_0_, product0_.price as price4_0_, product0_.quant
7 | Bottle | Accessories | 300.0 | 0
14 | Bottle | Accessories | 300.0 | 0
3 | Mouse | Electronics | 500.0 | 50
10 | Mouse | Electronics | 500.0 | 50
4 | Keyboard | Electronics | 1200.0 | 30
11 | Keyboard | Electronics | 1200.0 | 30
5 | Chair | Furniture | 3500.0 | 15
12 | Chair | Furniture | 3500.0 | 15
6 | Table | Furniture | 8000.0 | 5
13 | Table | Furniture | 8000.0 | 5
1 | Laptop |  HP | 50000.0 | 1
8 |  Laptop |  HP | 50000.0 | 1
2 | Laptop | Electronics | 55000.0 | 10
9 | Laptop | Electronics | 55000.0 | 10
```

```
===== PRODUCT MANAGEMENT (HQL LAB) =====
1. Insert Product
2. Insert Sample Products
3. View Product By ID
4. Sort Products By Price (ASC)
5. Sort Products By Price (DESC)
6. Sort Products By Quantity (High → Low)
7. Pagination (First 3 / Next 3)
8. Aggregate Functions
9. Filter By Price Range
10. LIKE Queries
11. Exit
Enter choice: 5
Hibernate: select product0_.id as id1_0_, product0_.description as descript2_0_, product0_.name as name3_0_, product0_.price as price4_0_, product0_.qua
2 | Laptop | Electronics | 55000.0 | 10
9 | Laptop | Electronics | 55000.0 | 10
1 | Laptop |  HP | 50000.0 | 1
8 |  Laptop |  HP | 50000.0 | 1
6 | Table | Furniture | 8000.0 | 5
13 | Table | Furniture | 8000.0 | 5
5 | Chair | Furniture | 3500.0 | 15
12 | Chair | Furniture | 3500.0 | 15
4 | Keyboard | Electronics | 1200.0 | 30
11 | Keyboard | Electronics | 1200.0 | 30
3 | Mouse | Electronics | 500.0 | 50
10 | Mouse | Electronics | 500.0 | 50
7 | Bottle | Accessories | 300.0 | 0
14 | Bottle | Accessories | 300.0 | 0
```

```
--- First 3 Products ---
Hibernate: select product0_.id as id1_0_, product0_.desc
1 | Laptop |  HP | 50000.0 | 1
2 | Laptop | Electronics | 55000.0 | 10
3 | Mouse | Electronics | 500.0 | 50

--- Next 3 Products ---
Hibernate: select product0_.id as id1_0_, product0_.desc
4 | Keyboard | Electronics | 1200.0 | 30
5 | Chair | Furniture | 3500.0 | 15
6 | Table | Furniture | 8000.0 | 5
```

```
Names starting with 'L':
Hibernate: select product0_.id as id1_0_, product0_.descr
1 | Laptop |  HP | 50000.0 | 1
2 | Laptop | Electronics | 55000.0 | 10
9 | Laptop | Electronics | 55000.0 | 10

Names ending with 'r':
Hibernate: select product0_.id as id1_0_, product0_.descr
5 | Chair | Furniture | 3500.0 | 15
12 | Chair | Furniture | 3500.0 | 15

Names containing 'top':
Hibernate: select product0_.id as id1_0_, product0_.descr
1 | Laptop |  HP | 50000.0 | 1
2 | Laptop | Electronics | 55000.0 | 10
8 |  Laptop |  HP | 50000.0 | 1
9 | Laptop | Electronics | 55000.0 | 10
```

```
Names with length = 5:
Hibernate: select product0_.id as id1_0_, product0_.descr
3 | Mouse | Electronics | 500.0 | 50
5 | Chair | Furniture | 3500.0 | 15
6 | Table | Furniture | 8000.0 | 5
10 | Mouse | Electronics | 500.0 | 50
12 | Chair | Furniture | 3500.0 | 15
13 | Table | Furniture | 8000.0 | 5

===== PRODUCT MANAGEMENT (HQL LAB) =====
1. Insert Product
2. Insert Sample Products
3. View Product By ID
4. Sort Products By Price (ASC)
5. Sort Products By Price (DESC)
6. Sort Products By Quantity (High → Low)
7. Pagination (First 3 / Next 3)
8. Aggregate Functions
9. Filter By Price Range
10. LIKE Queries
11. Exit
Enter choice: 11
Feb 06, 2026 10:25:16 PM org.hibernate.engine.jdbc.connec
INFO: HHH10001008: Cleaning up connection pool [jdbc:mysq
Exiting Application...
```

Git Repository:

https://github.com/Aravind2k6/2400030040-Fsad-WorkBook.git

VIVA QUESTIONS:

1. What is HQL and how does it differ from SQL?

A. HQL (Hibernate Query Language) is an object-oriented query language used in Hibernate to fetch data using entity classes.

Difference: HQL works with Java classes and properties, whereas SQL works with database tables and columns.

2. Why do we use sorting in HQL?

A. Sorting in HQL is used to arrange the result data in a specific order (ascending or descending).

It is done using the ORDER BY clause to improve data readability and organization.

3. What is pagination and where is it used?

A. Pagination is the process of dividing large data into smaller pages (limited records per page).

It is used in web applications to display data page by page and improve performance.

4. Name any three aggregate functions supported in HQL.

A. Some aggregate functions in HQL are:

- **count() – counts number of records**
- **sum() – calculates total value**
- **avg() – calculates average value**

5. What is the difference between HQL and HCQL (Hibernate Criteria Query Language)?

A. HQL is a string-based query language where we write queries using entity class names and properties, whereas HCQL (Hibernate Criteria Query Language) is a programmatic, object oriented approach where queries are created using Java methods instead of writing query strings.